Analysis of Sequence Modeling

Shivanshu Kumar
v1.0

Abstract

This document aims to capture the evolution of Deep Learning Techniques in Language Modeling. It is meant for folks entering the field of Language Modeling and Transformers. As someone who has gone through the process, I can relate to the initial intimidation when just starting out research in the area. Those will basic background in Machine Learning and Deep Learning will be greatly benefited, although if you sit with an LLM (whose surgery is the main theme in this doc) – you should be able to quickly catch up with unknown terminology.

Moreover, I have also tried to exhaustively tried to cite and describe briefly the relevant landmark papers and it might help you address your concern of "What papers do I start reading" ?

This document will evolve over several versions and currently discusses:

1. Sequence modeling as a problem of Artificial Intelligence
2. Fundamentals of Deep Learning methods for Language(Sequence) Modeling
3. Methods to make these methods efficient

In future version I intend to add:

1. How do these language transform into multi-purpose chat bots ?
2. Reasoning paradigms in and outside of language modeling
3. Multimodal models
4. Utility of transformers outside sequence modeling namely in vision and chem/bio-informatics
5. Relevant benchmarks and datasets, along with code guidelines to train your own models
6. Understanding how language models work under the hood and how we can use this knowledge to make them better and faster

Any feedback / comments are greatly appreciated! Please write to me at shivanshu.ydv@gmail.com

Contents

Part I

Evolution of Sequence Models

Generation 1 : $\mathcal{O}(1)$ hidden state

The task of sequence modeling involves producing outputs from a sequence of inputs of the same type - this output could in itself be a sequence. In the latter case the problem is termed as "Sequence-to-Sequence Modeling".

The problem of language modeling (generation) falls under this category since the task of auto-completion or conversation involves taking a text input of arbitrary size and then outputing another sequence of arbitrary size (length). Another feature of language is "context". Each sub-input carries its own context which might change drastically if some part of the input is ignored, hence to generate the output sequence the entire input should be kept in mind.

A common model to accomplish this is the **auto-regressive paradigm** wherein, given an input *prompt* $t_1, t_2 ... t_k$ of words, the model is aimed at producing

$$P(t_{k+1}|t_1 ... t_k)$$

Note that framing the problem in such a way makes it recursive, i.e., to generate the next word[1] we set input $= t_1, t_2 ... t_k$ and repeat the process n times. Hence a sequence $t_{k+1} ... t_{k+n}$ is obtained, which is the output of the model (Language generator). The underlying sequence-to-next-token module is a neural network.

Now given this auto-regressive framework where we can have arbitrary inputs of any length and all the tokens in the sequence contribute to the output, including the ones that are generated – one simple way is to store a "hidden state" representing the dynamic state of the generated sequence, where all the tokens add their contributions as they are generated.

Observe that the expression

$$(\prod_{i=2}^{n} P\ (\ t_i \mid t_{i-1}\ ...\ t_1\ )) \times P(t_1) = P(t_1\ ...\ t_n)$$

So iterating the token generation conditioned over the previous sub-sequence is essentially computing the joint probability of the entire sequence!

1.1 RNNs

The most vanilla model as mentioned above is a Recurrent neural network - named so, since the computation repeats over "steps". More formally, the Neural Network maintains a hidden state s_i at every time step i (corresponding to the i^{th} token in the sequence. With every incoming input token x_i, the state is updated as follows.

$$\begin{aligned} s_i &= (Ux_i + Ws_{i-1} + b) \\ y_i &= \sigma\left(Vs_i + c\right) \end{aligned} \tag{1.1}$$

Note that prior to any input the state s_0 could be set randomly or learned along with the weights U, V and W. y_i represents the output at time step i.

Suppose there is input sequence $x_1, x_2, ...x_k$, the outputs $y_1, ...y_k$ can be ignored and for $i > k$ we can set $x_i = y_{i-1}$, which clearly fits our auto regressive language generation paradigm.

Typical Neural Networks are trained via Gradient based Backpropagation algorithms - which require gradient computation of the defined loss function with respect to the weights of the network.

[1] in practice the granularity of language models is a sub-word aka a token

In order to compute loss on the entire sequence: we take the binary cross entropy loss at each time step, which is the probability of the predicted token:

$$L_t = -\log(P(y_t|x_1...x_t)$$

. The total loss being

$$L = \sum L_i$$

the sequence is typically truncated unless the model predicts $< eos >$ (End of Sequence) on its own. The derivative now becomes

$$\frac{\partial L}{\partial W} = \sum \frac{\partial L_i}{\partial W}$$

Note that the gradient computation is tricky, since for L_j,

$$\frac{\partial L_j}{\partial W} = \frac{\partial L_j}{\partial s_j}\frac{\partial s_j}{\partial W}$$

this comes from the chain rule – since L_j is a direct function of only s_j. So the computational dependencies of L_j on previous tokens is through state update rules of s_j which is captured in $s_j = Us_{j-1} + Wx_j + b$. Note that since s_{j-1} also depends on U, W, the gradient computation is non-trivial and involves a **back-propagation through time**. We employ the multiplication rule of derivatives to obtain :

$$\frac{\partial s_j}{\partial W} = \underbrace{\frac{\partial^+ s_j}{\partial W}}_{\text{explicit}} + \underbrace{\frac{\partial s_j}{\partial s_{j-1}}\frac{\partial s_{j-1}}{\partial W}}_{\text{implicit}}$$

For $j = 4$, the computation is unrolled and shown (Credits Prof. Mitesh Khapra's slides)

$$\begin{aligned}
&= \frac{\partial^+ s_4}{\partial W} + \frac{\partial s_4}{\partial s_3}[\underbrace{\frac{\partial^+ s_3}{\partial W}}_{\text{explicit}} + \underbrace{\frac{\partial s_3}{\partial s_2}\frac{\partial s_2}{\partial W}}_{\text{implicit}}] \\
&= \frac{\partial^+ s_4}{\partial W} + \frac{\partial s_4}{\partial s_3}\frac{\partial^+ s_3}{\partial W} + \frac{\partial s_4}{\partial s_3}\frac{\partial s_3}{\partial s_2}\left[\frac{\partial^+ s_2}{\partial W} + \frac{\partial s_2}{\partial s_1}\frac{\partial s_1}{\partial W}\right] \\
&= \frac{\partial^+ s_4}{\partial W} + \frac{\partial s_4}{\partial s_3}\frac{\partial^+ s_3}{\partial W} + \frac{\partial s_4}{\partial s_3}\frac{\partial s_3}{\partial s_2}\frac{\partial^+ s_2}{\partial W} + \frac{\partial s_4}{\partial s_3}\frac{\partial s_3}{\partial s_2}\frac{\partial s_2}{\partial s_1}\left[\frac{\partial^+ s_1}{\partial W}\right]
\end{aligned}$$

The above algorithm is extremely problematic for the same reason as very deep neural networks are, too many multiplications in a term leading to **vanishing or exploding** gradients.

This, along with the fact that RNNs weigh each token equally in the "context vector" which is essentially a $256 - 512$ dimensional vector leads to context pollution in practice, making them unfit for language modeling. Therefore we introduce **Gating mechanisms** to focus on the "important" tokens.

1.2 LSTMs and GRUs

In a **Long Short Term Memory** – do selective read and write into the hidden state by using a learned binary gate $o_i \in \{0, 1\}^n$ which masks the state for updation i.e., and a gated hidden output $h_t = o_t \odot s_t$ is used.

To update s_t, we use i_t, f_t which are fancily called the input and forget gates, updated using the previous state and current input for each time step:

$$\begin{aligned}
o_t &= \sigma\left(W_o h_{t-1} + U_o x_t + b_o\right) \\
i_t &= \sigma\left(W_i h_{t-1} + U_i x_t + b_i\right) \\
f_t &= \sigma\left(W_f h_{t-1} + U_f x_t + b_f\right)
\end{aligned}$$

Selective read :

$$s'_t = Uh_{t-1} + Wx_t + b$$

Selective write :

$$s''_t = i_t \odot s't$$

Selective forget (I never got why this exists):

$$s_t = s''_t + f_t \odot s_{t-1}$$

A variant is called the **Gated Recurrent unit** which eliminates f_t and uses $1 - i_t$ instead, and uses s_{t-1} directly for updating the gates :

$$o_t = \sigma\left(W_o h_{s-1} + U_o x_t + b_o\right)$$
$$i_t = \sigma\left(W_i h_{s-1} + U_o x_t + b_i\right)$$

$$s_t = i_t \odot s'_t + (\not{1} - i_t) \odot s_{t-1}$$

Note that the gradient flow in this case is gate-controlled and any time step not contributing is by-passed.

Problem with this is, once something is bypassed it is bypassed forever – for long context scenarios, say a word that is relevant for another is far behind in the sentence (Source : Google Gemini) :

> "The highly intellectual and renowned **professor**, who had authored numerous books and was admired by countless students, finally revealed **his** groundbreaking theory on the universe's origin."

the context for "his" is far behind sitting with the word "professor" – this typically happens in a lot of sequence modeling tasks. The fundamental limitation with these models is that they store sequence level information in constant space and even the state-of-the-arts were unable to support more than hundreds of tokens. Transformers, as discussed in the next chapter solve this problem by storing the information corresponding to each of the tokens separately.

Generation 2 : Transformers

This generation of sequence-to-sequence models store entire context , i.e., $\mathcal{O}(n)$ in the size of the sequence (n). Introduced in the seminal paper : Vaswani et al. 2017, Transformers have become the de-facto for Large Language Models, which are conversational agents built on top of decoder-only transformers, by fine-tuning them on instruction and conversation data. What we end up getting is an encoder decoder model, which could have two different modalities as input and output as long as the hidden state dimensions are the same.

Attention Mechanism The attention mechanism enables token generation by appropriately weighting the previous tokens in the stored context in terms of relevance. Formally, an attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

Vaswani et al. use "Scaled Dot-Product Attention". The input consists of queries and keys of dimension d_k, and values of dimension d_v. They compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values. The square root scaling is added for numerical scaling and works better in practice. The novelty of the attention mechanism is that the queries, keys and values of the entire sequence can be encoded as a matrix (dimension = $N \times d_{model}$). This enables incredible parallelism and acceleration using GEMM operations on general purpose GPUs.

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{2.1}$$

Multi-Head Attention Furthermore, linearly projecting the queries, keys and values h times with different, learned linear projections to d_k, d_k and d_v dimensions, respectively, helps the model learn richer features. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v-dimensional output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2.1.

$$\begin{aligned}\text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O \\ \text{where head}_\text{i} &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)\end{aligned}$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{model} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{model}}$.

N. Shazeer attempt to share a singly learn KV-head for all heads, while Ainslie et al. generalize this by grouping the KV-heads and sharing one vector per head, with minimal degradation in performance. These techniques have become standard and are widely used in practise.

Feed forward networks: In addition to attention sub-layers, each of the "transformer-blocks" contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{2.2}$$

Embeddings and Softmax: All the input and output are represneted as vectors in an embeddding space of dimension d_{model}. At the last layer the token representation is projected onto a vector of

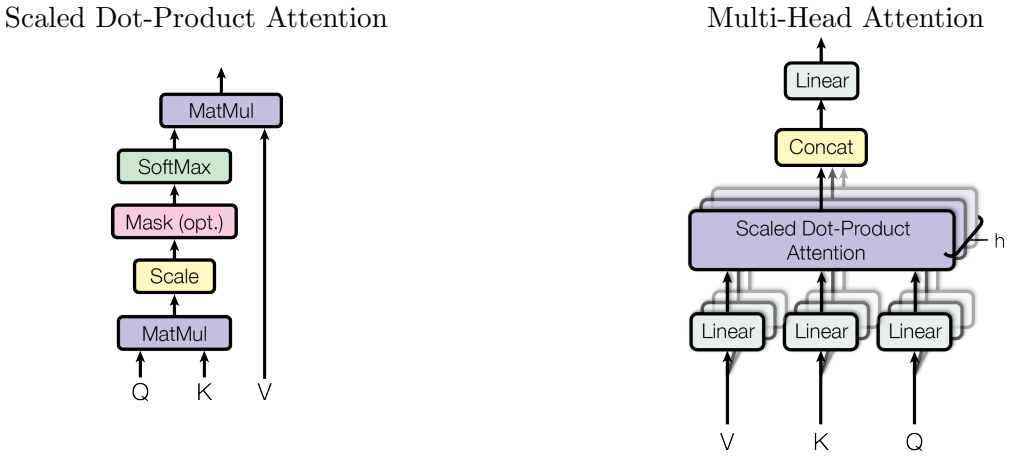


Figure 2.1: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

dimension of vocabulary size which is softmax-ed to get a probabilty distribution.

Positional Encoding: In order for the model to make use of the order of the sequence, information about the relative or absolute position of the tokens in the sequence is injected using "positional encodings" to the input embeddings. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. One way is to use sine and cosine functions of different frequencies:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$.

2.1 Encoder-Decoder Architecture

The original transformer architecture follows an encoder-decoder design for sequence-to-sequence tasks. Here, the encoder maps an input sequence of symbol representations $(x_1, ..., x_n)$ to a sequence of continuous representations $\mathbf{z} = (z_1, ..., z_n)$. Given $\mathbf{z}$, the decoder then generates an output sequence $(y_1, ..., y_m)$ of symbols one element at a time. At each step the model is auto-regressive, consuming the previously generated symbols as additional input when generating the next.

Encoder: The encoder is composed of a stack of identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network.

Decoder: The decoder is also composed of a stack of identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack.

The overall architecture of the model is residual, wherein each of the computational blocks adds its output to the input before passing over to the next block. See figure 2.2

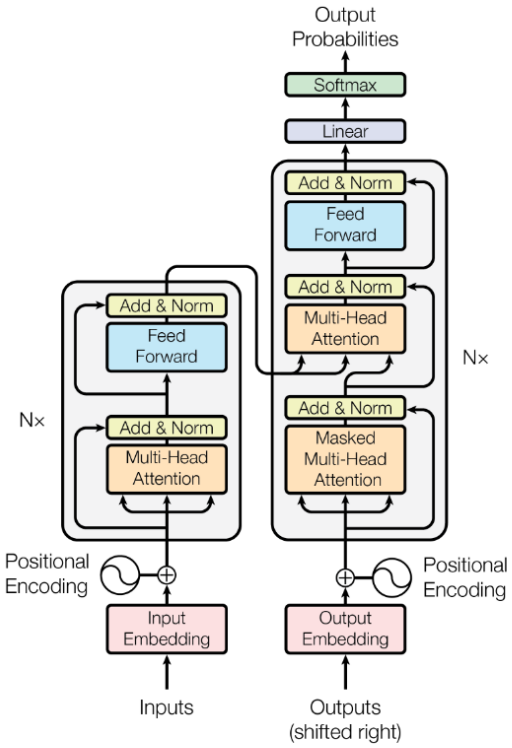


Figure 1: The Transformer - model architecture.

Figure 2.2: Encoder-Decoder Transformer

2.2 Decoder Only Architecture

Radford and Narasimhan 2018 introduced decoder-only transformers as a more-suited architecture for Natural-Language Generation. Scaling such models up to hundreds to billions of parameters gives us Large Language Models as we know them today.

The overall pipeline is: for the input token vector x, one hot at the token-id in the vocabulary,

$$x \to xE = e \text{ token embedding}$$

$$x \to xP = p \text{ positional encoding}$$

input representation of the token with position information encoded:

$$h_0 = p + e$$

for all the decoder blocks,

$$u_i = \text{LayerNorm}(h_i)$$

$$v_i = u_i + \text{Self-Attention}(u_i)$$

$$w_i = \text{LayerNorm}(v_i)$$

$$h_{i+1} = v_i + \text{MLP}(v_i)$$

(Note that MLP refers to the feed-forward network discussed above)

During training, loss is computed using cross-entropy loss function on the output probability vector $\hat{y}_t \in \mathbb{R}^V$ and the one-hot label for that position $y_t \in \{0,1\}^V$.

$$L = -\frac{1}{T}\sum_{t=1}^{T}\sum_{i=1}^{V} y_{t,i} \log \hat{y}_{t,i}$$

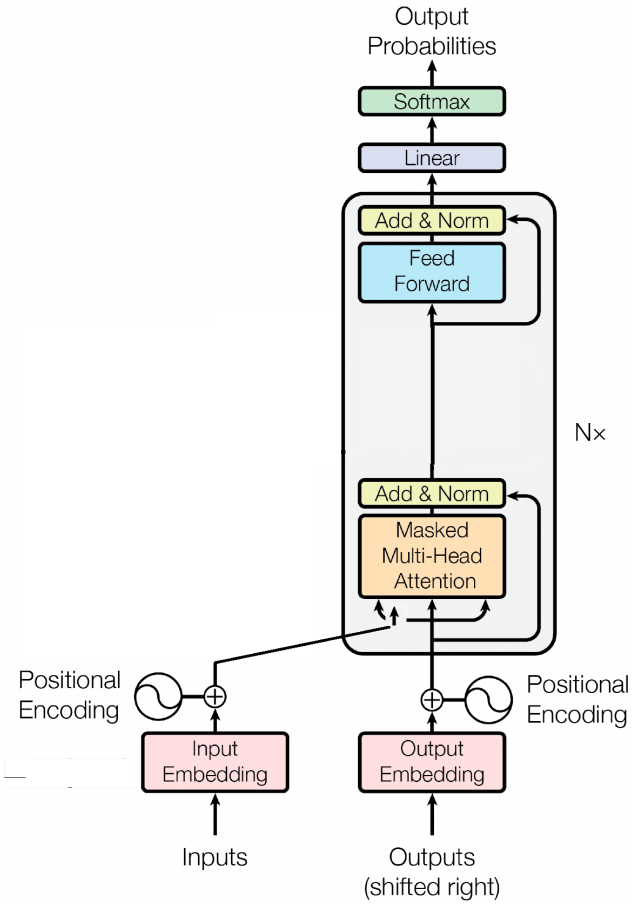


Figure 2.3: Decoder-only Tranformers

Since y_t is a one-hot vector this essentially is negative log-probability for the label token as predicted by our model.

$$L = -\frac{1}{T}\sum_{t=1}^{T} y_{t,i_t} \log \hat{y}_{t,i_t}$$

See Figure 2.3

In practice, a Transformer++ recipe is employed for high-performance training, which uses:

- **SWIGLU activations** (**shazeer2020gluvariantsimprovetransformer**): For input vector x (dimension d_{model}), we have weight matrices up-proj: we $W_u \in \mathbb{R}^{d_{model} \times d_{ff}}$, gate-proj: $W_g \in \mathbb{R}^{d_{model} \times d_{ff}}$ and down-proj $W_d \in \mathbb{R}^{d_{ff} \times d_{model}}$. Let b_u, b_g, b_d be the respective biases.

  The overall computation constitutes of the following steps:
  Project and gate:

  $$u = xW_u + b_u$$

  $$g = xW_g + b_g$$

  Apply element-wise activation (Swish) on u:

  $$\text{Swish}(u) = u \cdot \sigma(u)$$

  (where σ is the sigmoid function)
  Element-wise multiply gated and activated vectors:

  $$h = \text{Swish}(u) \odot g$$

Project back to model dimension:

$$\text{MLP}_{\text{SwiGLU}}(x) = hW_d + b_d$$

- **Rotary position encodings** (**rope**): This introduces position information by rotating the query and key vectors in multi-head attention instead of statically inputting the token representation with the position information encoded.

  For a query (or key) vector $x \in \mathbb{R}^d$, and token at position m, For each pair of dimensions $(2i, 2i+1)$ (with $i = 0, \ldots, d/2-1$):

  Let

  $$\theta_i = \text{base}^{-2i/d}$$

  (commonly, base = 10000), which acts as a frequency like in sinusoids.

  Define a 2×2 rotation matrix for each dimension pair:

  $$R_{m,i} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix}$$

  Apply this to every (even, odd) block of the input vector x:

  $$[x_{2i}, x_{2i+1}]^T \to R_{m,i}[x_{2i}, x_{2i+1}]^T$$

  For the whole vector:

  $$f(x, m) = R_m x$$

  Where R_m is the block-diagonal matrix made from $R_{m,i}$ blocks.

  Both query and key vectors are rotated by their respective positions and the dot product after rotation is:

  $$q_m'^T \cdot k_n' = q_m^T R_m^T R_n k_n$$

  This formulation ensures the attention depends on the relative position $m - n$

2.3 State of the Art Language Models

2.3.1 GPT

Generative Pre-trained Transformers introduced by Radford and Narasimhan, train a Langauge Model that is "pre-trained" on a self-supervised objective and further fine-tune (supervised) on labeled datasets for the task at hand, without changing the backbone model architecture. Utilize a decoder-only transformer architecture trained on a language generation objective. Key insight is that transformers trained on large unlabelled corpora can transfer knowledge across various tasks with little fine-tuning.

Note that GPT-2 was trained with dropout Srivastava et al. 2014 (To be disccused in a future version)

* **Google PaLM** is a 540 B parameter model (released before Gemini was introduced for multi-moadlity) with the aim for scaling params. Demonstrate efficient deployment on a large number of TPUs using "PathWay" technique designed for asynchronous distribution of compute. Use Parallel architecture : $y = x + \text{MLP}(\text{LayerNorm}(x)) + \text{Attention}(\text{LayerNorm}(x))$. Also utilize MQA N. Shazeer 2019.

* **Pythia** (Biderman et al. 2023) also uses parallel MLP and attention.

2.3.2 BERT

Bidirectional Encoder Representations Source: Devlin et al. 2019.

- Encoder-only model used for **Language understanding** or *Representation Learning*
- Jointly conditions both through the left and right by introducing bi-directional context and minimal architectural changes for downstream application (like GPT)
- As opposed to the auto-regressive models, use Masked Language Modeling - replace some tokens randomly with a mask and predict those tokens
- Pre-training objective: using two bidirectional models does not help : since when predicting the $t + 1^{th}$ token, t^{th} token has context from it, this becomes trivial for the model to "peek" into the answer before prediction.
- All the tokens self-attend to each other – refining representations through the layers and for final prediction etc, use the [CLS] token's final representation.
- For fine-tuning send the input-output pairs concatenated which is essentially cross-attention between the two!
- For measuring performance on say GLUE: use final representation of [CLS] and softmax through it
- ROBERTA Liu et al. 2019 improved BERT by (1) applying dynamic instead of static masking, (2) evaluating different model input formats and the disregarding the necessity of Next Sentence Prediction (NSP)(3) experimenting with larger batch sizes and Applying BPE text encoding, training with more data and performing optimal hyperparameter optimization.

In a future version the following model architectures will be discussed in detail: Qwen A. Yang et al. 2025, Gemma Team et al. 2024, DeepSeek DeepSeek-AI et al. 2025 and reasoning models.

2.3.3 LLaMA

Touvron et al. introduced a family of decoder-only transformer foundation models (7B–65B) which demonstrating competitive performance by training on large, publicly available corpora rather than proprietary datasets.

LLaMA adopts contemporary architectural choices that improve stability and length generalization (pre-normalization, SwiGLU-style feed-forward nonlinearity, and rotary positional embeddings RoPE, RMS normalisation), which together yield better training stability and handling of longer contexts.

2.3.4 Mistral

Jiang et al. further gave high capability models at small parameter counts by combining architectural simplifications with attention optimizations and inference-aware design including **GQA** (**gqa**) and sliding-window-attention.

Sliding-Window Attention (SWA) SWA limits per-layer attention to a local window while relying on stacked layers to propagate longer-range context; combined with rotating/limited KV caches this substantially reduces KV memory during long-context inference.

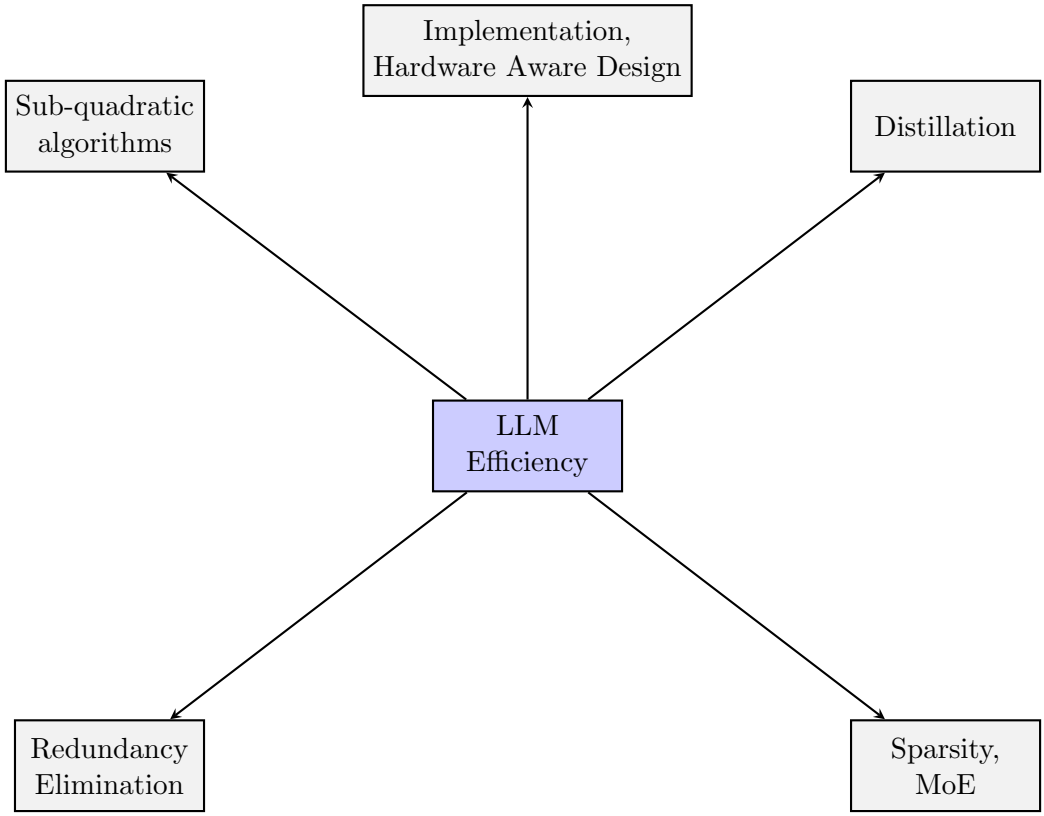
Mistral authors later released Mixtral (Mixtral of Experts), a Sparse Mixture-of-Experts variant (8 experts per layer) that exposes a very large parameter pool while keeping per-token compute small (router selects few experts), improving math, code and multilingual capability at reduced active compute.

Note: **Large** models like GPT-4o/5, Claude 4.0, Grok 4 and Gemini-2.5 are closed-source and little is known about the architectural developments brought into these models

Part II

Efficiency of Language Models

Research in systems optimization of LLMs



3.1 Distillation

Introduced by Hinton, Vinyals, and Dean 2015 in context of classification for neural networks. Distillation is one of the ways of making training faster for smaller models by utilizing "knowledge" from Large pre-trained models. Sources : C. Yang et al. 2024; Y. Gu et al. 2024. Logit based distillation typically involves optimizing over a loss function to maximize the similarity between the teacher and student model distribution

$$\mathcal{L} = KL\ (\ p_t\ ||\ p_s\ )$$

where p_t and p_s are the final probability distribution over the vocabulary, obtained after softmaxing the final output layer and $KL(x||y)$ is the Kullback-Leibler Divergence, defined over discrete distributions as

$$KL(x||y) = \sum_i x_i \log \frac{x_i}{y_i}$$

- Off-policy distillation : Force the output of the teacher on-to the student – the output is generated by the teacher model and the token are forced into the student model to get probability distributions, which are minimized to tune the weights Loss = $L_{off} = E_{x\sim D}[KL(P_t(y|x)||P_s(y|x))]$
- On-policy distillation : the student actively generates a response which is updated based on teacher feedback

3.2 Sparsity through MOE

N. M. Shazeer et al. 2017 introduce Mixture of Experts as a way to efficiently train heavily parametrized Neural Nets by training special MOE layers containing a bunch of expert layers, some of which are

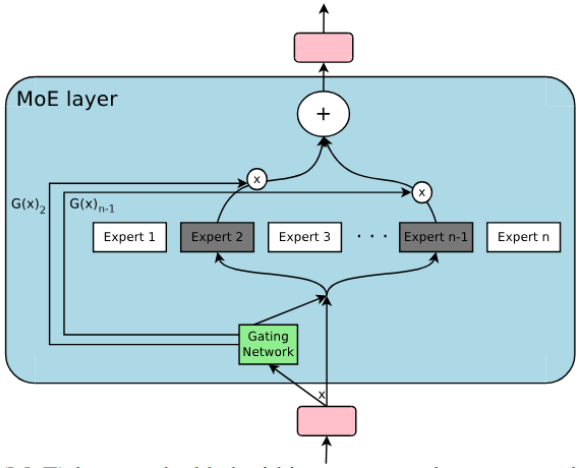


Figure 3.1: Mixture of Experts Layer

used for the final computation at inference by a separate gating layer. Training is done jointly with an auxiliary loss to make sure experts are not under-utilized. See Figure 3.1

Fedus, Zoph, and N. Shazeer 2022 introduced MOE in context of Transformer based Language models to route **MLP computation** for every token to a *single* expert. Qwen Mixtral Palm

3.3 Efficient Implementation

This involves designing hardware-aware algorithms for (1) Efficient GPU utilization (2) Striking the right tradeoff between compute and memory (3) Minimizing communication latency
The most prominent set of papers along this line is the Flash-Attention series Dao et al. 2022; Dao 2024; Shah et al. 2025:
Flash-Attention 1 The key observation in the naive implementation of Transformers is I/O is a huge bottleneck over FLOPS – so the authors attempt to push FLOPS up while reducing costly HBM accesses to arrive at a tradeoff that achieves an overall speedup. Figures 3.2 and 3.3 show the difference in implementations – the key observation is to break $\mathbf{K}, \mathbf{V}, \mathbf{Q}, \mathbf{O}$ into chunks so that the intermediate matrix $\mathbf{P} \in \mathbb{R}^{N \times N}$ is never stored in and hence transferred to the HBM N is typically $>> d$ and $\mathbf{P}$ wont fit in the SRAM. So a portion big enough to fit is computed on-chip, and its contribution is added to $\mathbf{O}$ appropriately, avoiding the costly I/O altogether! **Flash-Attention 2** improves FA by

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.
1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

Figure 3.2: Standard Attention

(1) reducing non-matmul FLOPs, which are much more costly than matmul FLOPs due to NVIDIA tensor cores ; (2) Parallelizing on the sequence alongside heads and batch; (3) better warp utilization in a thread-block (set of 32 warps) as shown in Figure 3.4). Since every block of the matrix $\mathbf{V}$ has to be multiplied by every block of $\mathbf{QK^T}$, sharing $\mathbf{V}$ across all warps minimizes communication latency and synchronization delays. **Flash-Attention 3** further improves performance specifically for H100 GPUs by utilizing its tensor memory accelerator and FP-8 precision.

Paged-Attention (Kwon et al. 2023) take inspiration from virtual memory in Operating Systems - instead of the common convention of storing Pytorch tensors in contiguous space that leads to an unnecessary memory allocation (fragmentation) - they build a paging system at the granularity of KV blocks. They also support KV sharing similar to (Copy-on-Write in OS processes), across several inference requests.

Algorithm 1 FlashAttention

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.
Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \ldots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \ldots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \ldots, m_{T_r}$ of size B_r each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  for $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
    On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
    Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
    Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
  end for
end for
Return $\mathbf{O}$.

Figure 3.3: Flash-Attention 1 implementation

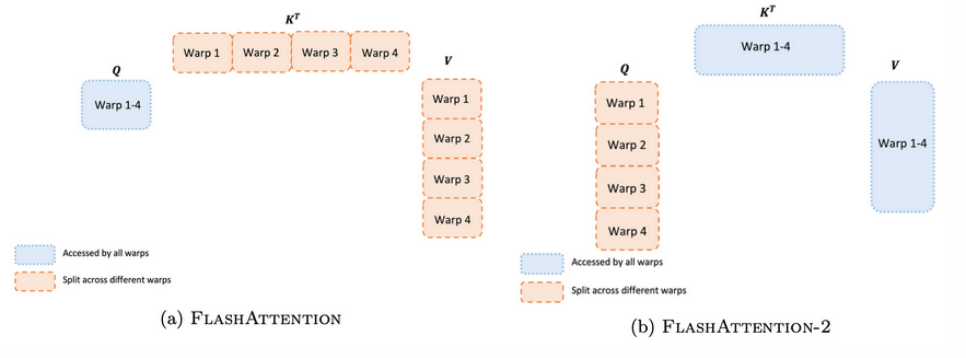


Figure 3.4: Flash Attention 2 Work Partitioning

3.4 Eliminating Redundancy

Frankle and Carbin 2019 propose that dense, randomly-initialized, feed-forward networks contain subnetworks (winning tickets) that—when trained in isolation— reach test accuracy comparable to the original network in a similar number of iteration. Neural networks like Transformers are heavily over-parametrized and have superflous parameters that can be eliminated. The following are some methods for efficient utilization of parameters:

3.4.1 Pruning

Pruning: Intoduced by **obd-lecun-etal** , pruning involves removing redundant parameters in the model. There are two major dimensions along which models are pruned:

- Width - individual weights, attention heads or embedding dimensions.
- Layer or depth pruning

Here we will discuss two methods which are most relevant:

1. **Weight pruning** involves removing redundant weights and avoiding the corresponding computation to achieve speed up and reduction in model size. Structured pruning involves removing entire structural components such as rows, columns, or blocks from weight matrices **ashkboos2024slicegptcompresslargelanguage**; **ma2023llmprunerstructuralpruninglarge**. On the other hand, unstructured pruning removes individual weights by setting them to zero, creating sparse matrices without following predetermined structural patterns **frantar2023sparsegptmassivelanguagemodels**; **sun2024simpleeffectivepruningapproach**.

Such pruning methods typically require specialized hardware to achieve maximum benefit. For example, NVIDIA tensor cores offer support for 2:4 structured sparsity but necessitate large batch sizes to obtain meaningful gains in performance **mishra2021acceleratingsparsedeepneural**; Song et al. 2024.

2. **Block pruning** involves identifying redundant blocks (decoder layers) in the network at inference time using either an importance or redundancy metric, eliminating the less important (or more redundant) layers, and subsequently fine-tuning the compressed model to recover performance. Elimination of entire blocks not only lowers parameter count but also reduces the memory overhead associated with maintaining KV cache, which scales linearly with the sequence length.

   men2024shortgpt quantify the layer importance based on expected cosine similarity between the input and output of a block, and prune blocks with low scores. Gromov et al. also use cosine similarity, but remove an entire block of consecutive layers, followed by fine-tuning using Quantization and Low Rank Adapters (QLoRA). Y. Yang, Cao, and Zhao replace consecutive layers with a single layer by appropriately setting the parameters of the replaced layer. On the other hand, **siddiqui2024deeperlookdepthpruning** use a Shapley value-based metric measured on a calibration dataset for quantifying block importance. Song et al. iteratively prune blocks by evaluating the perplexity on a calibration set and remove the most redundant block at each step. However, these methods retain the high costs of pre-training while incurring additional overhead for layer selection and post-training optimization.

From a practical standpoint Muralidharan et al. 2024 describe best practices for compact LLM deployment using pruning and distillation.

3.4.2 KV-sharing and elimination

Seeking to reduce KV cache requirements, prior works have attempted to share key-value pairs across attention layers. brandon2024reducing proposed sharing KV pairs across adjacent layers in groups of two, reducing the KV cache size by roughly 50%. **sun2024cacheoncedecoderdecoderarchitectures** also compress the KV cache by 50% by computing KV pairs only for the first half of the model and sharing them for the remaining layers. **rajput2024inferencefriendlymodelsmixattention** place sliding window attention layers sandwiched between global attention layers, sharing keys and values across the global layers and between consecutive sliding window layers. H. Wu and Tu further compress the KV cache by computing keys and values only in the top layers and pairing them with queries in the bottom layers.

3.4.3 Quantization

Quantization reduces numerical precision of model parameters and activations to lower memory footprint and computational cost while maintaining accuracy. For large language models, two families of methods are prominent: post-training (inference-only) quantization and quantization-aware or mixed-precision training.

Post-training quantization (PTQ): Weight-only quantization techniques compress pre-trained models without retraining. **frantar2023gptqaccurateposttrainingquantization** introduce GPTQ, which performs second-order, layer-wise quantization of transformer weights, enabling 3–4 bit inference with minimal accuracy loss. **xiao2022smoothquant** propose SmoothQuant, which smooths activation outliers to support practical INT8 (W8A8) deployment. **lin2024awq** further refine this with activation-aware weight quantization (AWQ), preserving critical weights for stable 4-bit inference. These methods typically achieve 2–4× memory reduction and 1.5–3× inference speedup depending on kernel support.

Quantized fine-tuning and mixed precision: dettmers2023qlora propose QLoRA, which fine-tunes low-rank adapters on 4-bit quantized backbones, allowing efficient adaptation of large models on a single GPU. During training, mixed-precision techniques use FP16 or BF16 arithmetic for most operations while retaining FP32 master weights for stability.

The key insight is that reducing precision does not harm performance too much!

3.5 Sub-quadratic Algorithms

A different line of research aims to reduce the time and memory complex- ity of transformers by proposing various algorithmic enhancements. During auto-regressive generation, the attention mechanism exhibits quadratic computational complexity $\mathcal{O}(T^2)$ and linear memory com- plexity $\mathcal{O}(T)$ with respect to sequence length T . For each newly generated token, the query vector must compute attention scores with the key vectors of all preceding tokens, while the KV cache stores all the previous key-value pairs to avoid recomputation This topic has been discussed in detail in Section 4.1

Generation 3

In the entire chapter we following notation is used:
d is in the input dimension or embedding dimension
T is the length of the input sequence both during training and inference
W_α represents a weight matrix

4.1 State Space Models

State Space Models are a general class of continuous sequence to sequence models. For one dimensional sequences continuous in time, $x(t), y(t) \in \mathbb{R}$, there is a hidden state sequence $\{h\}, h(t) \in \mathbb{R}^N$.

$$\begin{aligned} \frac{dh}{dt} &= Ah(t) + Bx(t) \\ y(t) &= Ch(t) \end{aligned}$$

Since our sequences are discrete sequences, the discrete version is given by

$$\begin{aligned} h_t &= Ah_{t-1} + Bx_t \\ y_t &= Ch_t \end{aligned} \tag{4.1}$$

Notice the similarity and differences between 1.1 and 4.1 (of course there is no bias term here) – the structure is the same *but the hidden state is in a higher dimension!*, so when we represent tokens by our embedding vectors $\in \mathbb{R}^d$, h_t would $\in \mathbb{R}^{N \times d}$. This raises the dimension of A to a rank-3 tensor.

The formulation in 4.1 and formulations that look this are called **linear recurrence forms** (We will see how transformers reduce to this form!). An alternate formulation is **global convolution** :

$$\begin{aligned} h_1 &= Bx_1 \text{ assuming } h_0 = 0 \\ h_2 &= ABx_1 + Bx_2 \\ h_3 &= A^2Bx_1 + ABx_2 + Bx_3 \end{aligned}$$

the output is $y_t = CA^{t-1}Bx_1 + CAt - 2Bx_2 + \ldots + CBx_t$
Convolving $CA^{n-1}B, CA^{n-2}, \ldots CB$ with $x_n, x_{n-1} \ldots x_1$ gives the vector $(y_{2n}, y_{2n-1} ... y_n ... y_1)$. The sequence of our interest is $(y_n, ..., y_1)$ Hence this is equivalent to the the convolution $K * x$ with a well defined kernel K. Note that this formulation does not hold when the parameters of the model are *time (or input) dependent.*

4.1.1 Mamba

A. Gu and Dao 2024 Mamba is a state space model as described in 4.1 except that here the parameters A, B, C are no longer time-independent. For the input x the parameters B, C are dependent on the input according to $\text{Linear}_N(x)$.

The exact layer structure of the Mamba architecture is shown in Figure 4.1. The input goes through a linear transformation followed by a depth-wise one convolution given by

$$o_{i,d} = \sum_{j=0}^{k-1} W_{j,d} \cdot x_{i-j,d} + b_d$$

which followed by an activation goes through the SSM block discussed above. Finally the two projections are multiplied and brought back to the input dimension of the model.

Since the equivalence between convolution and mamba does not hold and there is no equivalent matrix kernel form as in 4.2, the authors propose hardware-aware algorithms by exploiting the memory hierarchy of GPUs for efficient training. Inference is linear in the sequence length and memory requirements do not scale; making Mamba excellent at handling long-context sequences. Also, note that

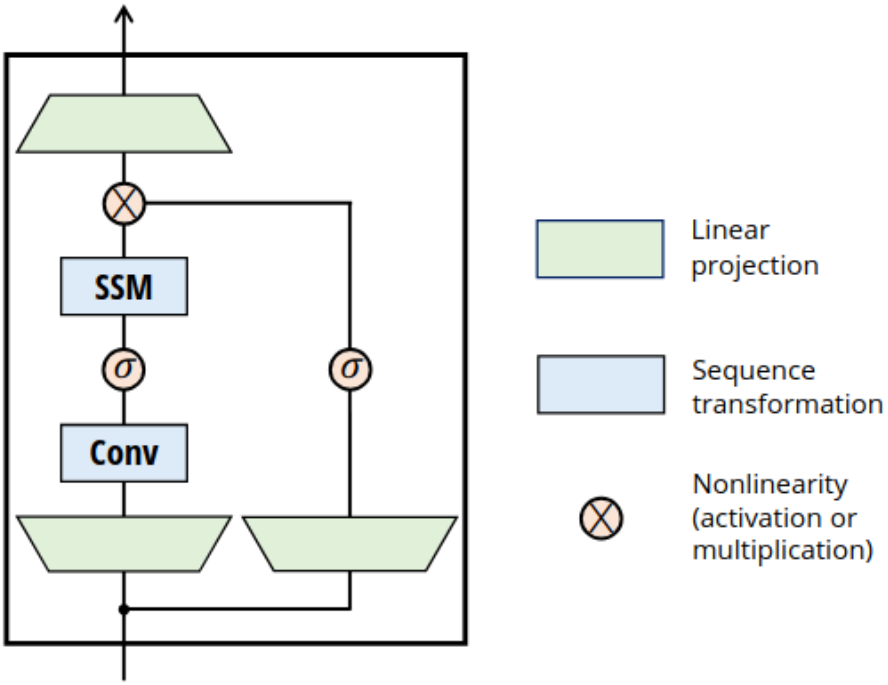


Figure 4.1: Mamba Architecture

even though LSTMs try to fit selectivity in context storage by introducing gating. Although state space models are a different formulation from RNNs but as proved by A. Gu and Dao gated RNNs are a special case of structured SSMs.

The key to deploying SSMs is the equivalent convolution formulation that helps massively parallelize training - a significant advantage over RNN-based models.

Note: Despite higher efficiency, architectures with constant-size hidden state representations suffer from recency bias, where recent tokens disproportionately influence the hidden state. This makes it hard to capture long-range dependencies. **ro2025dsla** maintain both linear and quadratic attention versions for every layer. An attention-score entropy metric dynamically decides which version to use at inference time, balancing computational cost and accuracy. However, this requires costly pre-training along with distillation to train the linear versions of each layer.

4.2 Linear Attention

Katharopoulos et al. 2020 establish the equivalence between a linear transformer (without softmax activation on the Query-Key product) and RNNs (Note that the matrices here are transposed as opposed to the original paper) Assuming X is the input sequence $X \in \mathbb{R}^{d \times T}$ and $Q, K, V \in \mathbb{R}^{d \times d}$, the output of the Self-Attention module (unmasked) is given by :

$$
\begin{aligned}
Q &= W_Q X \\
K &= W_K X \\
V &= W_V X \\
A_l(X) = V' &= \text{softmax}\left(\frac{QK^T}{\sqrt{D}}\right) V.
\end{aligned}
$$

The softmax operation makes things tricky since it is a per-row computation that cannot be done

independently before doing the matrix multiplication.

An equivalent formulation would be

$$V_i' = \frac{\sum_{j=1}^{T} \text{sim}\left(Q_i, K_j\right) V_j}{\sum_{j=1}^{T} \text{sim}\left(Q_i, K_j\right)}.$$

Thinking of sim as a kernel operation between Q_i and K_j,

$$V_i' = \frac{\sum_{j=1}^{T} \phi\left(Q_i\right)^T \phi\left(K_j\right) V_j}{\sum_{j=1}^{T} \phi\left(Q_i\right)^T \phi\left(K_j\right)},$$
$$\Rightarrow V_i' = \frac{\phi\left(Q_i\right)^T \sum_{j=1}^{T} \phi\left(K_j\right) V_j^T}{\phi\left(Q_i\right)^T \sum_{j=1}^{T} \phi\left(K_j\right)}$$

Now things become nicer, since during auto-regressive generation, what we want from the previous tokens is their keys and values. So for the i^{th} token during inference,

$$S_i = \sum_{j=1}^{i} \phi\left(K_j\right) V_j^T$$
$$Z_i = \sum_{j=1}^{i} \phi\left(K_j\right)$$
$$V_i' = \frac{\phi\left(Q_i\right)^T S_i}{\phi\left(Q_i\right)^T Z_i}.$$

Now, S_i and Z_i can be computed from S_{i1} and Z_{i1} in constant time!

Also observe the stark similarity with an RNN here :

$$s_0 = 0$$
$$z_0 = 0$$
$$s_i = s_{i-1} + \phi\left(W_K x_i\right)\left(W_V x_i\right)^T$$
$$z_i = z_{i-1} + \phi\left(W_K x_i\right)$$
$$y_i = f_l\left(\frac{\phi\left(W_Q x_i\right)^T s_i}{\phi\left(W_Q x_i\right)^T z_i} + x_i\right)$$

During training, when the entire sequence is available we can use the mat-mul form to compute in parallel, **during inference** the complexity naturally comes down to linear, and this *gives us the best of both worlds in RNNs and Transformers!*

4.3 Diffusion Language Models

As discussed in 1, the objective of language modeling is to model the distribution over token sequences, $P\left(t_1 ... t_k\right)$. This is achieved in the auto-regressive paradigm by *iteratively* generating tokens conditioned on the context and then adding the token to the context. This paradigm is inherently sequential in the sequence dimension. Taking inspiration from diffusion models in image generation (Sohl-Dickstein et al. 2015), researchers have proposed language generation using the diffusion paradigm.

Auto-regressive models are **probabilistic models** that are easier to train and sample, while diffusion or energy-based models provide much more flexibility, although sampling becomes hard. The idea is to start with noisy tokens distributed according to say a Gaussian and then *denoise* each token to model the entire sequence's probability distribution. This enables parallelism in generation.

- During training, a **forward diffusion process** introduces noise in text tokens to map them to continuous noise over t time steps, although in case of language this becomes challenging since *text representations (embeddings) are discrete vectors* [1]
- For prediction during training and inference, the model applies a learnt **reverse diffusion** process to replace a subset of noisy tokens with text and over a number of time-steps the entire sequence is retrieved.

Although diffusion models provide flexibility and bidirectional context knowledge and parallel generation, Following are some disadvantages:

1. Diffusion process from a continuous to discrete space
2. Training instability
3. Computational inefficiency due to large number of denoising steps for quality in generation

Despite these challenges below are some of the SOTA diffusion models:

- Nie et al. 2025 Scale diffusion models to 8B params.
- Gemini Diffusion – Google's diffusion language model
- Labs et al. 2025 have released a free-playground coding model at `https://www.inceptionlabs.ai/introducing-mercury`. Also utilize Transformer architecture (and hence the efficient implementations) and develop efficient kernels and inference engines specific for the diffusion process

Wu et al. 2025 optimize diffusion LLMs by introducing KV-Cache and Parallel Decoding similar to how Flash-Attention 1,2,3 accelerate LLMs (Dao et al. 2022; Dao 2024; Shah et al. 2025) using efficient kernel implementation and hardware-aware algorithmic modification

4.4 Energy-Based Models

Energy-Based Models (EBMs) define an implicit probability distribution over outputs by associating each input–output pair (x, y) with a scalar energy $E(x, y) \in \mathbb{R}$, where lower energy implies higher compatibility. The model's inference objective is to find

$$\hat{y} = \arg\min_{y \in Y} E_\theta(x, y),$$

which corresponds, under the Boltzmann formulation, to the conditional probability

$$P(y|x) = \frac{e^{-E_\theta(x,y)}}{\int e^{-E_\theta(x,y)} dy}.$$

This energy minimization process implicitly performs both generation and verification: generation emerges through gradient descent in the energy landscape, while verification corresponds to evaluating the energy of candidate outputs.

Training aims to shape the energy surface so that correct pairs occupy low-energy regions while negatives lie higher. Early approaches used contrastive learning—lowering energy for positive samples and raising it for negatives—but were computationally inefficient. More recent regularized learning formulations enforce smoothness and stability in the energy landscape without explicit contrastive sampling.

EBMs offer several advantages relevant to efficient and general-purpose modeling. They do not require normalized probabilities, enabling flexible reuse of computation across tasks. Their formulation naturally accommodates multimodality and uncertainty by assigning comparable energies to multiple

[1] This means that the set of embeddings that make "sense" is a finite and very sparse subset of $\mathbb{R}^{\text{vocab-size}}$, as opposed to pixel data, where intensity values are continuous

plausible outputs, rather than collapsing them into a single prediction. Moreover, the optimization-based inference allows task-agnostic reasoning and dynamic computation, properties that make EBMs conceptually aligned with system-2 style processing in language models.

Recent work interprets diffusion models as a subclass of EBMs, where the energy corresponds to the denoising score at each diffusion step, highlighting EBMs as a unifying framework for both generative and discriminative modeling **lecun2022ebm**, http://atcold.github.io/NYU-DLSP20/. Although inference through iterative energy minimization remains computationally heavy, EBMs provide an important direction toward verification-driven and data-efficient alternatives to direct likelihood modeling in large language systems.

4.5 Hybrid Models

Hybrid models aim to combine State-Space models and Quadratic transformer blocks into the same model (Some models also combine Diffusion and auto-regressive models) Some works in these direction include:

- Convolutional Hybrid Models:Ku et al. 2025
- Dual-State Linear Attention Ro et al. 2025
- LLaMA-ZEBRA M. Yang et al. 2025

4.6 Scaling laws for Language models

Kaplan et al. 2020 Hoffmann et al. 2022 Scaling laws show that language-model performance follows simple power-law trends with model size, dataset size and compute: increasing any of those improves cross-entropy predictably until diminishing returns set in Kaplan et al. 2020. Hoffmann et al. refined this by deriving compute-optimal tradeoffs (solve for the best model size vs. tokens under a fixed compute budget). They established that data (in tokens) $\approx 20\times$ parameters is compute optimal.

However, this objective disregards the inference budget, which becomes critical when serving a language model at scale. In this context, given a target level of performance, the preferred model is not the fastest to train but the fastest at inference, and although it may be cheaper to train a large model to reach a certain level of performance, a smaller one trained longer will ultimately be cheaper at inference. Touvron et al. show this by obtaining improved performance when their 7B model is trained on 1T tokens. Jiang et al. further use better quality data and obtain better performance, despite using GQA(Ainslie et al. 2023) and Sliding-window attention. These works show that the compute optimality problem is 3 dimensional : model capabilities, training cost, inference cost.

Part III

LLMs as Chat-bots

Post-training

// chapters from here onwards need significant improvement and will be updated in future versions

Foundation models like GPT, Claude and Gemini are language generation models and cannot simulate conversations well – well in the sense that they are not *meant* for conversation - they are meant for "yapping" given some prefill initialization (prompt as a more fancy term)

A typical pipeline in deploying production grade "chat / conversation" models, starting from the "foundation" language generation models is shown below:

Figure 5.1: Training pipeline.

Bibliography

Ainslie, J. et al. (Dec. 2023). "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Ed. by H. Bouamor, J. Pino, and K. Bali. Singapore: Association for Computational Linguistics, pp. 4895–4901. DOI: 10.18653/v1/2023.emnlp-main.298. URL: https://aclanthology.org/2023.emnlp-main.298/.

Biderman, S. et al. (2023). "Pythia: a suite for analyzing large language models across training and scaling". In: *Proceedings of the 40th International Conference on Machine Learning*. ICML'23. Honolulu, Hawaii, USA: JMLR.org.

Dao, T. (2024). "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning". In: *The Twelfth International Conference on Learning Representations*. URL: https://openreview.net/forum?id=mZn2Xyh9Ec.

Dao, T. et al. (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". In: *Advances in Neural Information Processing Systems*. Ed. by S. Koyejo et al. Vol. 35. Curran Associates, Inc., pp. 16344–16359. URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/67d57c32e20fd0a7a302cb81d36e40d5-Paper-Conference.pdf.

DeepSeek-AI et al. (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv: 2501.12948 [cs.CL]. URL: https://arxiv.org/abs/2501.12948.

Devlin, J. et al. (June 2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by J. Burstein, C. Doran, and T. Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, pp. 4171–4186. DOI: 10.18653/v1/N19-1423. URL: https://aclanthology.org/N19-1423/.

Fedus, W., B. Zoph, and N. Shazeer (Jan. 2022). "Switch transformers: scaling to trillion parameter models with simple and efficient sparsity". In: *J. Mach. Learn. Res.* 23.1. ISSN: 1532-4435.

Frankle, J. and M. Carbin (2019). *The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks*. arXiv: 1803.03635 [cs.LG]. URL: https://arxiv.org/abs/1803.03635.

Gromov, A. et al. (2025). "The Unreasonable Ineffectiveness of the Deeper Layers". In: *The Thirteenth International Conference on Learning Representations*. URL: https://openreview.net/forum?id=ngmEcEer8a.

Gu, A. and T. Dao (2024). "Mamba: Linear-Time Sequence Modeling with Selective State Spaces". In: *First Conference on Language Modeling*. URL: https://openreview.net/forum?id=tEYskw1VY2.

Gu, Y. et al. (2024). "MiniLLM: Knowledge Distillation of Large Language Models". In: *The Twelfth International Conference on Learning Representations*. URL: https://openreview.net/forum?id=5h0qf7IBZZ.

Hinton, G. E., O. Vinyals, and J. Dean (2015). "Distilling the Knowledge in a Neural Network". In: *ArXiv* abs/1503.02531. URL: https://api.semanticscholar.org/CorpusID:7200347.

Hoffmann, J. et al. (2022). *Training Compute-Optimal Large Language Models*. arXiv: 2203.15556 [cs.CL]. URL: https://arxiv.org/abs/2203.15556.

Jiang, A. Q. et al. (2023). *Mistral 7B*. arXiv: 2310.06825 [cs.CL]. URL: https://arxiv.org/abs/2310.06825.

Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models*. arXiv: 2001.08361 [cs.LG]. URL: https://arxiv.org/abs/2001.08361.

Katharopoulos, A. et al. (2020). "Transformers are RNNs: fast autoregressive transformers with linear attention". In: *Proceedings of the 37th International Conference on Machine Learning*. ICML'20. JMLR.org.

Ku, J. et al. (2025). *Systems and Algorithms for Convolutional Multi-Hybrid Language Models at Scale*. arXiv: 2503.01868 [cs.LG]. URL: https://arxiv.org/abs/2503.01868.

Kwon, W. et al. (2023). "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: SOSP '23. Koblenz, Germany: Association for Computing Machinery, pp. 611–

626. ISBN: 9798400702297. DOI: 10.1145/3600006.3613165. URL: https://doi.org/10.1145/3600006.3613165.

Labs, I. et al. (2025). *Mercury: Ultra-Fast Language Models Based on Diffusion*. arXiv: 2506.17298 [cs.CL]. URL: https://arxiv.org/abs/2506.17298.

Liu, Y. et al. (2019). "RoBERTa: A Robustly Optimized BERT Pretraining Approach". In: *ArXiv* abs/1907.11692. URL: https://api.semanticscholar.org/CorpusID:198953378.

Muralidharan, S. et al. (2024). "Compact Language Models via Pruning and Knowledge Distillation". In: *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. URL: https://openreview.net/forum?id=9U0nLnNMJ7.

Nie, S. et al. (2025). *Large Language Diffusion Models*. arXiv: 2502.09992 [cs.CL]. URL: https://arxiv.org/abs/2502.09992.

Radford, A. and K. Narasimhan (2018). "Improving Language Understanding by Generative Pre-Training". In: URL: https://api.semanticscholar.org/CorpusID:49313245.

Ro, Y. et al. (2025). "On-the-Fly Adaptive Distillation of Transformer to Dual-State Linear Attention for Long-Context LLM Serving". In: *Forty-second International Conference on Machine Learning*. URL: https://openreview.net/forum?id=pqHWzviKKN.

Shah, J. et al. (2025). "FlashAttention-3: fast and accurate attention with asynchrony and low-precision". In: *Proceedings of the 38th International Conference on Neural Information Processing Systems*. NIPS '24. Vancouver, BC, Canada: Curran Associates Inc. ISBN: 9798331314385.

Shazeer, N. (2019). *Fast Transformer Decoding: One Write-Head is All You Need*. arXiv: 1911.02150 [cs.NE]. URL: https://arxiv.org/abs/1911.02150.

Shazeer, N. M. et al. (2017). "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer". In: *ArXiv* abs/1701.06538. URL: https://api.semanticscholar.org/CorpusID:12462234.

Sohl-Dickstein, J. et al. (July 2015). "Deep Unsupervised Learning using Nonequilibrium Thermodynamics". In: *Proceedings of the 32nd International Conference on Machine Learning*. Ed. by F. Bach and D. Blei. Vol. 37. Proceedings of Machine Learning Research. Lille, France: PMLR, pp. 2256–2265. URL: https://proceedings.mlr.press/v37/sohl-dickstein15.html.

Song, J. et al. (2024). "SLEB: streamlining LLMs through redundancy verification and elimination of transformer blocks". In: *Proceedings of the 41st International Conference on Machine Learning*. ICML'24. Vienna, Austria: JMLR.org.

Srivastava, N. et al. (2014). "Dropout: A Simple Way to Prevent Neural Networks from Overfitting". In: *Journal of Machine Learning Research* 15.56, pp. 1929–1958. URL: http://jmlr.org/papers/v15/srivastava14a.html.

Team, G. et al. (2024). *Gemma: Open Models Based on Gemini Research and Technology*. arXiv: 2403.08295 [cs.CL]. URL: https://arxiv.org/abs/2403.08295.

Touvron, H. et al. (2023). *LLaMA: Open and Efficient Foundation Language Models*. arXiv: 2302.13971 [cs.CL]. URL: https://arxiv.org/abs/2302.13971.

Vaswani, A. et al. (2017). "Attention is All you Need". In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

Wu, C. et al. (2025). *Fast-dLLM: Training-free Acceleration of Diffusion LLM by Enabling KV Cache and Parallel Decoding*. arXiv: 2505.22618 [cs.CL]. URL: https://arxiv.org/abs/2505.22618.

Yang, A. et al. (2025). *Qwen3 Technical Report*. arXiv: 2505.09388 [cs.CL]. URL: https://arxiv.org/abs/2505.09388.

Yang, C. et al. (Oct. 2024). "Survey on Knowledge Distillation for Large Language Models: Methods, Evaluation, and Application". In: *ACM Trans. Intell. Syst. Technol.* ISSN: 2157-6904. DOI: 10.1145/3699518. URL: https://doi.org/10.1145/3699518.

Yang, M. et al. (May 2025). "Zebra-Llama: Towards Extremely Efficient Hybrid Models". In: *CoRR* abs/2505.17272. URL: https://doi.org/10.48550/arXiv.2505.17272.

Yang, Y., Z. Cao, and H. Zhao (Nov. 2024). "LaCo: Large Language Model Pruning via Layer Collapse". In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. Ed. by Y. Al-Onaizan, M. Bansal, and Y.-N. Chen. Miami, Florida, USA: Association for Computational Linguistics, pp. 6401–6417. DOI: 10.18653/v1/2024.findings-emnlp.372. URL: https://aclanthology.org/2024.findings-emnlp.372/.